

Experiment-2A

Aim: Develop a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.

Description:

Routing is the mechanism used in web applications to determine how an application responds to a client request for a specific URL and HTTP method (GET, POST, PUT, DELETE, etc.). A **route** consists of:

- **URL path** (e.g., /users)
- **HTTP method** (e.g., GET, POST)
- **Handler function** (logic that executes when the route is matched).

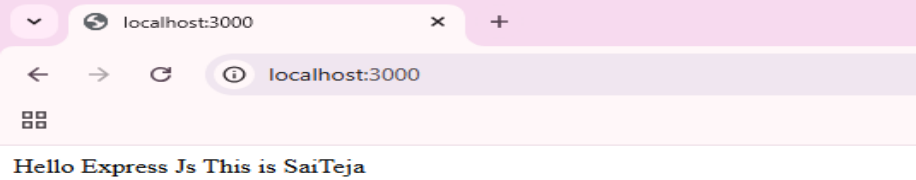
Route handling refers to how the application processes a matched route. **Route parameters** are **dynamic values** embedded directly in the URL path. **Query parameters** are optional key-value pairs appended to the URL after a ?. **URL building** is the process of **constructing URLs dynamically** using route paths, parameters, and query strings.

Program:

```
const express=require('express');
const app=express();
const port=5000;
app.get('/',(req,res)=>{
  res.send('Hello Express');
})
app.get('/login',(req,res)=>{
  res.send('Login Successfully created');
})
app.get('/login/123',(req,res)=>{
  res.send('Login id:123 Successfully created');
})
app.get('/student/:sno', (req, res) => {
  res.send(`Student Number: ${req.params.sno}`);
})
app.get('/search', (req, res) => {
  res.send(req.query);
})
app.get('/urlinfo', (req, res) => {
  constfullUrl = `${req.protocol}://${req.get('host')}${req.originalUrl}`;
  res.send(`You accessed: ${fullUrl}`);
})
app.listen(port,()=>{
  console.log('Server Started 5000 port');
})
```

Output:

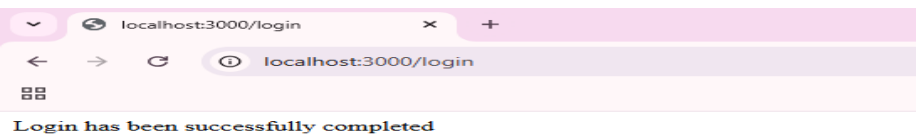
```
E:\>cd 24485A0503\Experiment-2  
  
E:\24485A0503\Experiment-2>node app  
Server Started 3000 port
```



localhost:3000

localhost:3000

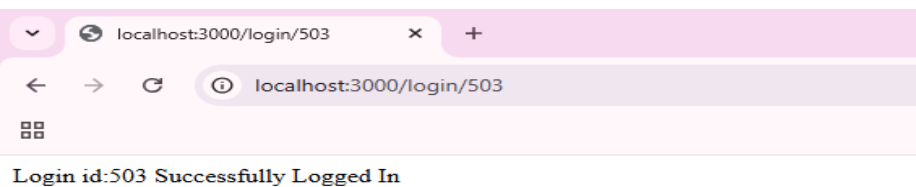
Hello Express Js This is SaiTeja



localhost:3000/login

localhost:3000/login

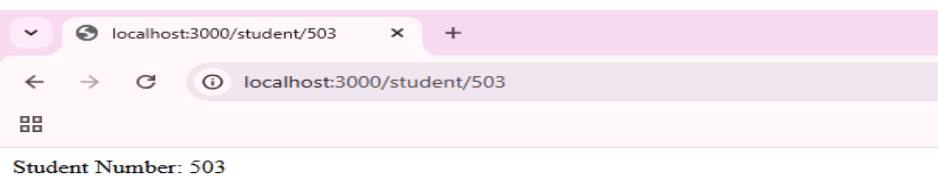
Login has been successfully completed



localhost:3000/login/503

localhost:3000/login/503

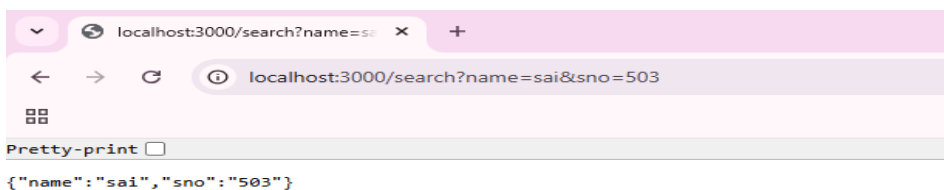
Login id:503 Successfully Logged In



localhost:3000/student/503

localhost:3000/student/503

Student Number: 503

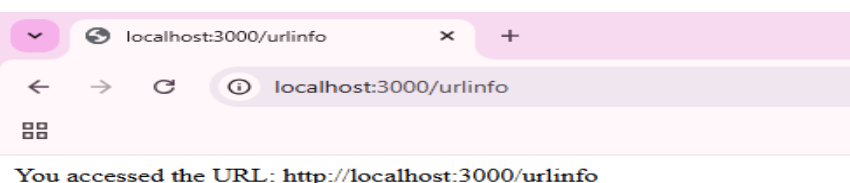


localhost:3000/search?name=sai&sno=503

localhost:3000/search?name=sai&sno=503

Pretty-print ☐

```
{ "name": "sai", "sno": "503" }
```



localhost:3000/urlinfo

localhost:3000/urlinfo

You accessed the URL: <http://localhost:3000/urlinfo>

Experiment-2B

Aim: Develop a program to accept data, retrieve data and delete a specified resource using http methods.

Description:

HTTP methods are used to perform different operations on resources in a web application. The **POST** method is used to accept data from the client and create a new resource on the server.

The **GET** method is used to retrieve data from the server without modifying it.

The **DELETE** method is used to remove a specified resource from the server.

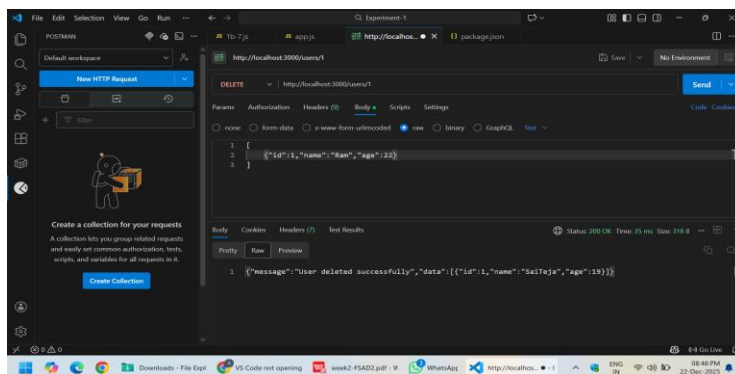
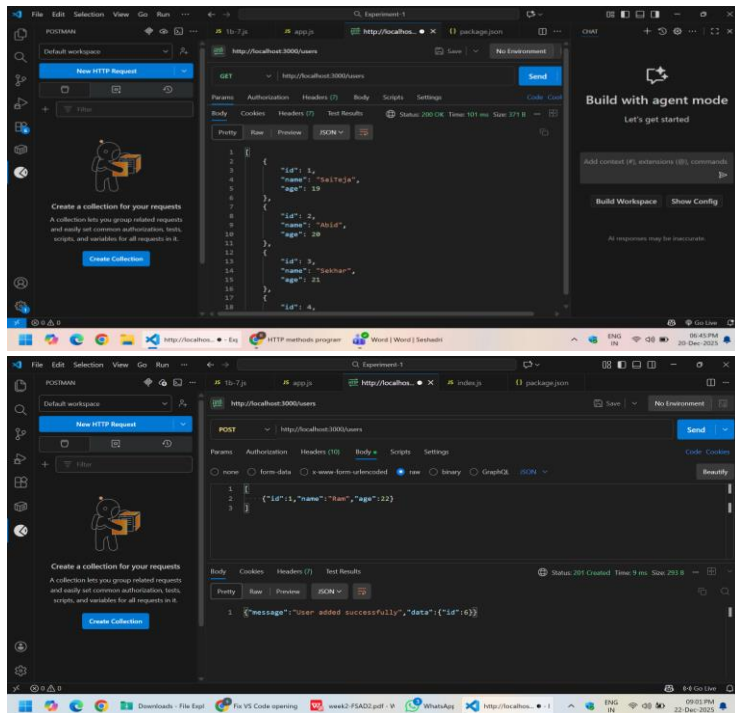
Each **HTTP** method defines a clear action, making applications structured, secure.

Program:

```
const express = require('express');
const app = express();
const PORT = 3000;
// Middleware to parse JSON data
app.use(express.json());
// In-memory data storage
let users = [
  { id: 1, name: "SaiTeja", age: 19 }, { id: 2, name: "Abid", age: 20 },
  { id: 3, name: "Sekhar", age: 21 }, { id: 4, name: "Krish", age: 22 }];
// GET – Retrieve data
app.get('/users', (req, res) => {
  res.status(200).json(users);
});
// POST – Accept / Insert new data
app.post('/users', (req, res) => {
  const newUser = {
    id: users.length + 1,
    name: req.body.name,
    age: req.body.age
  };
  users.push(newUser);
  res.status(201).json({
    message: "User added successfully",
    data: newUser
  });
});
// DELETE – Delete a specified resource
app.delete('/users/:id', (req, res) => {
  const userId = parseInt(req.params.id);
  const index = users.findIndex(user => user.id === userId);
```

```
if (index === -1) {
    return res.status(404).json({ message: "User not found" });
}
const deletedUser = users.splice(index, 1);
res.status(200).json({
    message: "User deleted successfully",
    data: deletedUser
});
});
// Start the server
app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
});
```

Output:



Experiment-2C

Aim: Write a program to show the working of middleware.

Description:

Middleware is a function that sits between the client request and the server response. It has access to:

- Request object
- Response object
- Next function (used to pass control to the next middleware)

Middleware processes requests before they reach the final route handler or after the response is sent.

Working of Middleware (Request–Response Cycle)

1. A client sends an HTTP request to the server.
2. The request enters the middleware chain.
3. Each middleware function executes in the order it is defined.
4. Middleware can:
 - End the request-response cycle, or
 - Pass control to the next middleware using a special mechanism.
5. Finally, the request reaches the route handler, which sends the response.

Program:

```
const express = require('express');
const app = express();
const port = 3000;
app.use((req, res, next) => {
  console.log(`[${req.method}] ${req.url} - Time: ${new Date().toLocaleTimeString()}`);
  next(); // pass control to the next middleware or route
});
const checkSecret = (req, res, next) => {
  if (req.query.secret === '1234') {
    next(); // allowed
  } else {
    res.send('Access Denied: Secret key missing or incorrect');
  }
};
app.get('/', (req, res) => {
  res.send('Welcome to Home Page!');
});
app.get('/secure', checkSecret, (req, res) => {
  res.send(' Access Granted to Secure Page!');
});
app.listen(port, () => {
  console.log(` Server running at http://localhost:${port}`);
});
```

});

Output:

```
E:\24485A0503\Experiment-2>node app
Server running at http://localhost:5000
|
```

